

Server Module — Technical Documentation

This document describes the `Server` class of the `ft_irc` project: the component responsible for the network layer — creating the listening socket, accepting clients, multiplexing I/O with a single `poll()`, buffering data, and tearing connections down cleanly.

Scope: this documents `Server.cpp` / `Server.hpp` only. The `Server` collaborates with `Client`, `Channel`, `Parser`, `CommandHandler`, and `Replies`; those are described here only at the level of *how* `Server` uses them.

1. Overview

The server is **single-threaded** and **event-driven**. It never forks and never blocks on a single client. One `poll()` call watches every file descriptor at once — the listening socket plus every connected client — and the program only touches a descriptor after `poll()` reports it is ready. This satisfies the subject's hard requirement: *only one `poll()` (or equivalent) for all read, write, listen operations*.

Lifecycle at a glance:

```
main → Server(port, password)           // parse + validate args
      → ServerInit()                     // socket, bind, listen,
register fd
      → ServerRun()                       // poll loop until _running ==
false
      → ~Server()                         // close everything, free
memory
```

2. Data members

Member	Type	Purpose
<code>_port</code>	<code>unsigned short</code>	Validated listening port (1024-65535).
<code>_password</code>	<code>const std::string</code>	Connection password required by clients.
<code>_serverSocket</code>	<code>int</code>	The listening socket fd.
<code>_pollFds</code>	<code>std::vector<struct pollfd></code>	Every fd watched by <code>poll()</code> . Index 0 is the listening socket.
<code>_clients</code>	<code>std::map<int, Client *></code>	Connected clients keyed by their fd.
<code>_channels</code>	<code>std::map<std::string, Channel *></code>	Active channels keyed by name.
<code>_running</code>	<code>static bool</code>	Loop flag, flipped to <code>false</code> by the signal handler.

`_running` is **static** so the C-style signal handler (which cannot receive `this`) can reach it. It is defined once outside the class and initialised to `true`.

3. Construction & validation

```
cpp
```

```
Server(const std::string &port, const std::string &password);
```

The port string is parsed through a `std::stringstream`:

- `extractInt.fail()` catches non-numeric input (e.g. `"abc"`).
- `!extractInt.eof()` catches trailing junk (e.g. `"80x"` or `"80 "`).
- A range check rejects anything outside `1024-65535`.

On any failure the constructor throws `std::invalid_argument`, so `main` should construct the `Server` inside a `try` block. Ports below 1024 are rejected by design to avoid privileged ports (which would require root to bind).

The destructor closes the listening socket, then closes every client fd and `delete`s every `Client` and `Channel`, and clears both maps. It is the single owner of all heap allocations and all open descriptors.

4. Network setup — `ServerInit()`

Returns `true` on success, `false` (with a message to `stderr`) on any failure.

1. `socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)` — IPv4, a reliable connection-oriented byte stream, TCP transport.
 2. `setsockopt(..., SO_REUSEADDR, ...)` — lets the program re-bind the port immediately after a restart. Without it the OS keeps the port in `TIME_WAIT` for a while and the next `bind()` fails with *"Address already in use."*
 3. **Fill** `struct sockaddr_in`
 - `sin_family = AF_INET`
 - `sin_addr.s_addr = htonl(INADDR_ANY)` — listen on every local interface (`0.0.0.0`).
 - `sin_port = htons(_port)` — host-to-network byte order.
 - `htonl`/`htons` exist because the network expects big-endian; the host may be little-endian.
 4. `bind()` — attach the socket to the address/port. `sockaddr_in` is cast to the generic `sockaddr *` because that is what `bind()` accepts.
 5. `listen(_serverSocket, SOMAXCONN)` — mark the socket passive and set the backlog.
 6. **Register** the listening fd in `_pollFds` with `events = POLLIN`. From now on, readiness on this fd means *a new connection is waiting*.
-

5. The event loop — `ServerRun()`

cpp

```
while (_running) {  
    int eventTrack = poll(&_pollFds[0], _pollFds.size(), 5000);  
    ...  
    for (int i = (int)_pollFds.size() - 1; i >= 0; i--) { ... }  
}
```

- `poll()` **returns a count, not a list.** It tells you *how many* fds have events, not *which* ones, so the loop must scan `_pollFds` and inspect each `.revents`.
- `revents` **is a bitmask.** Each flag (`POLLIN`, `POLLOUT`, ...) is one bit, so readiness is tested with bitwise AND: `if (curRev & POLLIN)`. A single `revents` can carry more than one flag at once, which is why each is checked independently.
- **Backwards iteration** (`i` from high to low) is deliberate: `cleanClosure()` erases entries from `_pollFds` during the loop. Going downward means an erase only shifts elements at *higher* indices, which have already been handled this pass.
- **Cached locals** `curFd` and `curRev` are read once at the top of each iteration, so later mutations to `_pollFds` cannot invalidate them.
- **Return values:**
 - `< 0` with `errno == EINTR` → interrupted by a signal; `continue` (do not crash).
 - `< 0` otherwise → real error; break out of the loop.
 - `== 0` → the 5000 ms timeout elapsed with no events; `continue`.

Dispatch per ready fd:

Condition	Action
<code>POLLIN</code> on the listening socket	<code>acceptNewClient()</code>
<code>POLLIN</code> on a client socket	<code>receiveClientData(curFd)</code>
<code>POLLOUT</code> on a known client	<code>sendMessage(curFd)</code>

The `POLLOUT` branch re-checks that the fd is still in `_clients`, because a `POLLIN` handler on the same pass may have already disconnected it.

6. Accepting connections — `acceptNewClient()`

- `accept(_serverSocket, ...)` returns a brand-new fd dedicated to one client. The `clientAddr` struct is passed so the kernel can fill in the peer's IP and port; it is cast to `sockaddr *` to match `accept()`'s signature.
- A `Client` object is allocated and its hostname is resolved from the numeric address with `inet_ntop()`.
- The new client is registered in **two** places that must always stay in sync:
 - `_clients[clientSocket]` — the lookup map.
 - `_pollFds` — a new entry with `events = POLLIN`.

Any fd added to `_clients` must also be added to `_pollFds`, and vice versa; the two collections are the single source of truth for "who is connected."

7. Receiving & buffering — `receiveClientData()`

```
cpp
ssize_t bytesRead = recv(clientFd, buff, sizeof(buff) - 1, MSG_DONTWAIT)
```

- `MSG_DONTWAIT` makes this one call non-blocking regardless of the socket's mode (a safety net when the fd is not put in non-blocking mode via `fcntl`).
- `bytesRead == 0` → the peer closed the connection cleanly → `cleanClosure()`.
- `bytesRead < 0` → currently treated as an error → `cleanClosure()`.

After a successful read the bytes are appended to the client's **input buffer**, and `Parser::extractLines()` pulls out every *complete* line. Each complete line is parsed into a `Message` and dispatched to `CommandHandler::handleCommand()`. If a command sets the client's "should close" flag (e.g. `QUIT`), the connection is closed and the loop over the remaining lines breaks.

Packet reassembly contract. Because TCP is a stream, one `recv()` may contain a partial command, multiple commands, or a command split across several reads (the subject's `nc com^Dman^Dd` test). Correct behavior depends on two invariants:

1. The input buffer **accumulates** across calls rather than being overwritten.
 2. `extractLines()` returns only lines terminated by `\n`/`\r\n` and leaves any trailing incomplete fragment in the buffer for the next read.
-

8. Sending — `switchPollOut()` and `sendMessage()`

Output is queued in each client's **output buffer**. Writing follows the *level-triggered* `poll()` model:

1. When there is data to send to a client, `switchPollOut()` sets that fd's events to `POLLIN | POLLOUT`.
2. On the next loop pass `poll()` reports `POLLOUT` (socket is writable) and `sendMessage()` runs.
3. `send()` may transmit only part of the buffer. The sent bytes are erased from the front and the fd stays in `POLLOUT` so the rest goes out on a later pass.
4. Once the buffer is empty, events drop back to `POLLIN` alone so `poll()` stops waking for writability on an idle socket.

This is why writes are never done directly inside command handling — everything goes through the buffer + `POLLOUT` so a slow client can never block the server.

9. Disconnection — `cleanClosure()`

Tear-down order matters and is:

1. `removeClientFromAllChannels()` — *while the client still exists*, so the QUIT message can be built from its nickname/username/hostname and broadcast.
2. `close(clientFd)`.
3. `delete` the `Client` and `erase` it from `_clients`.
4. Erase the matching entry from `_pollFds`.

Doing the channel cleanup first is essential — once the `Client` is deleted, its identity is gone and no correct QUIT could be sent.

10. Channel-wide quit — `removeClientFromAllChannels()`

1. If the client was registered, build a QUIT message via `Replies::QUIT_MSG(...)`.
 2. Collect, into a `std::set<Client *>`, every *other* member of every channel the client belonged to. A set is used so a client sharing several channels with the leaver receives the notice only once.
 3. Queue the QUIT message to each recipient's output buffer and flip them to `POLLOUT`.
 4. Remove the leaver from every channel's member / operator / invite lists.
 5. Delete any channel that is now empty (`erase(it++)`), the C++98-safe map-erase idiom).
-

11. Signals & shutdown — `signalHandler()`

A static handler sets `_running = false`. The next `poll()` either returns normally or is interrupted (`EINTR`), handled as `continue`), the `while` (`_running`) condition then fails, the loop exits, and `~Server()` releases every descriptor and allocation. This gives a clean shutdown on `Ctrl-C` with no leaked fds or memory.

12. Build & run

```
bash  
  
make  
./ircserv <port> <password>  
# example  
./ircserv 6667 mypassword
```

Compiled as C++98 with `-Wall -Wextra -Werror`. No external or Boost libraries.

13. Known considerations / verify before defense

These are points worth confirming or tightening; several depend on files outside this module:

- `send()` and `EAGAIN`. With `MSG_DONTWAIT`, a full kernel send buffer makes `send()` return `-1` / `errno == EAGAIN` (or `EWOULDBLOCK`). That is transient, not fatal — the current code disconnects on it. Handle `EAGAIN` / `EWOULDBLOCK` by returning and retrying on the next pass (this is the "low bandwidth" case the subject's test section calls out).
- **Input/output buffer semantics.** Confirm that the client's input and output buffer setters *append* rather than overwrite — otherwise split commands are lost and queued output can be clobbered by a QUIT broadcast.
- **Embedded NUL bytes.** Building a `std::string` from a `char[]` stops at the first `\0`. Constructing from `(buff, bytesRead)` preserves the exact byte count.
- **Listening socket mode.** `accept()` is not protected by `MSG_DONTWAIT`. Setting the fds non-blocking with `fcntl(fd, F_SETFL, O_NONBLOCK)` (the only form the subject allows) removes the rare blocking-`accept` edge case.
- **Allocation failure.** `new Client(...)` throws on failure rather than returning null; wrap it in `try` / `catch` if you want to guarantee "no crash when out of memory."
- **Orthodox Canonical Form.** `Server` owns raw pointers and fds; declaring the copy constructor and `operator=` private-and-undefined documents that it must not be copied.